

1. First, what is the problem?

The statement says:

One intentional issue I identified is data leakage during scaling. The notebook fits the scaler on the entire dataset before splitting into training and testing data. This means information from the future test period is used when calculating the scaling parameters. For a proper time-series pipeline, I would first split the data chronologically, fit the scaler only on the training data, and then use that same scaler to transform both training and test data.

We should never allow future/test data to influence how we prepare the training data.

For time-series data, this is especially important because the test data represents the future.

What does "scaling" mean?

Scaling means changing the numerical values of your data to a smaller, consistent range so that a machine-learning model can work with the features more effectively.

Notes: Scaling = putting numerical features onto a similar range.

Fitting the scaler = learning the minimum/maximum (or other statistics) needed for that transformation.

Example: 1. Imagine our holiday data looks like this

Suppose we have:

Date	Holiday	Sales
Jan 1	New Year	1,000
Feb 14	Valentine's Day	2,000
Mar 1	None	1,500
Apr 10	Easter	3,000
May 1	Labor Day	5,000

Let's say we're using **Sales** as a feature.

The values are:

1000

2000

1500

3000

5000

These numbers can be quite large, so we can scale them.

2) What calculation does **MinMaxScaler** do?

The formula is:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Python's MinMaxScaler performs this calculation.

Here:

Minimum = 1,000

Maximum = 5,000

For the New Year sales of 1,000:

$$\frac{1000 - 1000}{5000 - 1000} = 0$$

For Valentine's Day:

$$\frac{2000 - 1000}{5000 - 1000} = 0.25$$

For March:

$$\frac{1500 - 1000}{5000 - 1000} = 0.125$$

For Easter:

$$\frac{3000 - 1000}{5000 - 1000} = 0.50$$

For Labor Day:

$$\frac{5000 - 1000}{5000 - 1000} = 1$$

So we get:

Holiday	Sales	Scaled Sales
New Year	1,000	0.000
Valentine's Day	2,000	0.250
None	1,500	0.125
Easter	3,000	0.500
Labor Day	5,000	1.000

So Pythong did all the calculations for us:

What does fit() actually do?

important for understanding your data-leakage question.

When you write:

```
scaler.fit(data)
```

Python looks at the data and **learns the minimum and maximum**.

For example:

Training data:

1000

2000

1500

3000

Python learns:

min = 1000

max = 3000

It remembers those numbers.

Then:

```
scaler.transform(data)
```

uses those learned numbers to scale the data.

4. What does `fit_transform()` mean?

This: `scaler.fit_transform(data)`

Is basically:

```
scaler.fit(data)
```

```
scaler.transform(data)
```

So:

FIT → learn min/max

TRANSFORM → use min/max to scale

data-leakage issue

Suppose your holiday dataset is a **time series**.

Imagine:

2019 — 2020 — 2021 — 2022

Training data

Test data

If the code does this:

```
scaler = MinMaxScaler()
```

```
daily[feature_columns] = scaler.fit_transform(  
    daily[feature_columns]  
)
```

This is where the problem can happen like: because we are doing

```
fit_transform(ALL DATA)
```

before the train/test split data.

so python looks at

2019

2020

2021

2022

and finds the minimum and maximum across **everything**.

That means the scaler has seen information from the future/test period.

Incorrect data pipeline:

Python



```
# 1. Load data
df = pd.read_csv("holidays_events.csv")

# 2. Prepare data
# ...

# 3. Scale ALL data
scaler = MinMaxScaler()

df[feature_columns] = scaler.fit_transform(
    df[feature_columns]
)

# 4. THEN split
train = df[...]
test = df[...]
```

🚨 Potential data leakage occurs in Step 3, because `fit_transform()` sees the complete dataset before the split.

Correct pipeline is :

Python

```
# 1. Load data
df = pd.read_csv("holidays_events.csv")

# 2. Prepare data
# ...

# 3. Split chronologically
train = df[df['date'] < '2022-01-01']
test = df[df['date'] >= '2022-01-01']

# 4. Create scaler
scaler = MinMaxScaler()

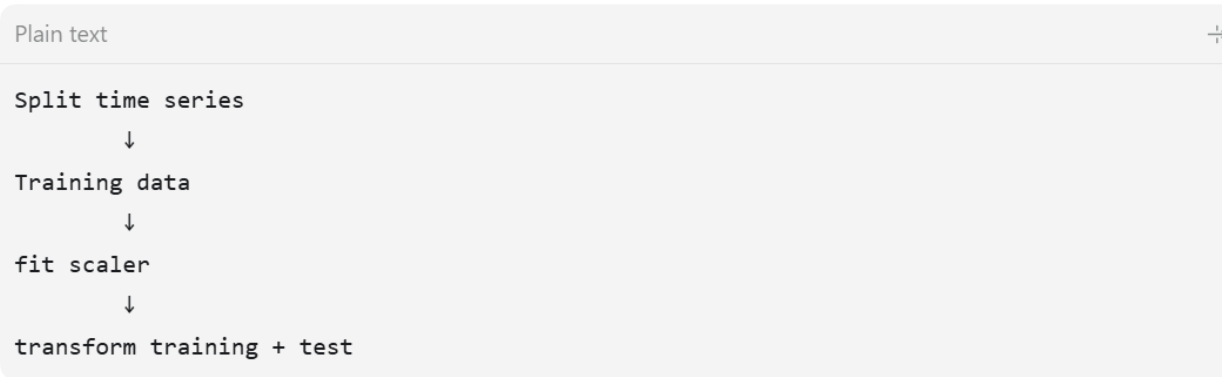
# 5. FIT ONLY on training data
scaler.fit(train[feature_columns])

# 6. Transform training data
train_scaled = scaler.transform(train[feature_columns])

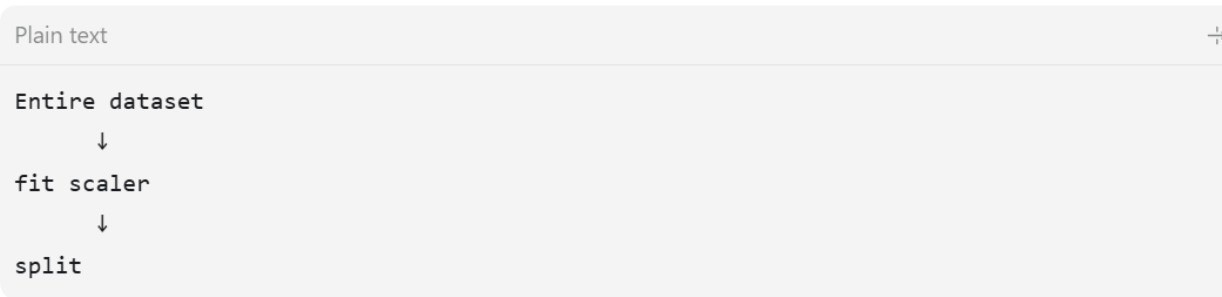
# 7. Transform test data
test_scaled = scaler.transform(test[feature_columns])
```

Better approach

Conceptually:



not:



Issue	Why it matters	How to improve
1. Scaling before train/test split	Test/future information influences scaler	Fit scaler only on training data
2. Validation approach	Time-series validation should preserve chronology	Create explicit chronological validation set
3. Label encoding categorical variables	Numeric labels can imply an artificial order	One-hot encoding/appropriate representation may be better
4. Averaging encoded categories	avg_type and avg_locale don't have a strong natural numerical meaning	Consider better categorical/time-series features
5. Very limited dataset	Daily aggregation may produce relatively few observations	More historical data would make deep learning more meaningful
6. RNN has no EarlyStopping	RNN trains all 50 epochs and could overfit	Add EarlyStopping similar to LSTM
7. Test set is used only once, which is good	But model-selection decisions should rely on validation, not test	Keep test set untouched until final evaluation